P1 — SECURE BANKING CLI

High-Level Design (HLD) · Low-Level Design (LLD) · Domain-Driven Design (DDD)

Architecture · Database · UML · Security · Transaction Integrity · Domain Model

Item	Detail
Project type	Individual, standalone, menu-driven command-line banking application (P1, Weeks 2–3)
Technology	Core Java 17+, JDBC, MySQL 8+ or PostgreSQL, Maven, JUnit 5, SLF4J/Logback, bcrypt (or Argon2id / PBKDF2)
Scope	Registration, login, profile, multiple accounts (Savings, Current, Deposit, Loan), deposit, withdrawal, atomic transfer, loan repayment, history, inactivity timeout, logout, audit logging, automated tests
Audience	Training participants, trainers, reviewers and evaluators
Document version	2.0 — 26 October 2026 (consolidates and supersedes HLD/DLD/UML v1.0)

Scope note. This is an educational banking simulation, not a production platform. It teaches secure password storage, authorization, SQL safety, session expiry, auditability and database transaction integrity. No real money, KYC, payment network or internet-facing API is involved.

Document control

Version	Date	Description
1.0	26 Oct 2026	Original blueprint and HLD / DLD / UML pack.
2.0	26 Oct 2026	Consolidated and corrected HLD, LLD (renamed from DLD) and new DDD part; schema, rules and diagrams revised (see table below).

Conventions

PK = primary key, FK = foreign key, UQ = unique, VO = value object, DAO = Data Access Object, DTO = Data Transfer Object, HLD / LLD / DDD = High-Level / Low-Level / Domain-Driven Design, CLI = command-line interface. Requirement IDs **P1-01…P1-14** come from the project brief; **F01…F14** are the design-level requirements in this document. Diagrams are conceptual artifacts; implementation detail may be refined as the code evolves.

What changed in version 2.0

#	Issue in v1.0	Resolution in v2.0
1	`CHECK (balance >= 0)` applied to every account, which conflicts with Loan semantics.	Loan balance is defined as *outstanding principal* (amount owed, never negative); loan terms live in `loan_details`; deposit terms in `deposit_details`.
2	State machine showed CREATED and MATURED, but the schema status CHECK allowed only ACTIVE/FROZEN/CLOSED.	Persisted statuses: ACTIVE, FROZEN, MATURED, CLOSED. CREATED is an in-memory state only.
3	Ledger design left open (one row vs paired entries).	Decision recorded: **one ledger row per business transaction** with optional source/destination and balance-after snapshots for reconciliation.
4	Audit of failed financial attempts undecided; only `login_audit` existed (brief P1-13 needs transaction events).	Added `transaction_audit`. Failed or rolled-back attempts are audited there and never written to the ledger as SUCCESS.
5	Deposit / Loan business rules were left to the participant.	Simplified rules fixed (term deposit, loan repayment) so that all four account types are testable.
6	Lockout, brute-force and timing considerations only hinted at.	Added `failed_login_count / locked_until`, generic errors and dummy-hash verification for unknown users.
7	Deadlock handling stated as a requirement but not designed.	Deterministic lock order, bounded retry on deadlock/lock-timeout, explicit isolation level, timeouts documented.
8	No domain-level design.	New Part C (DDD): ubiquitous language, bounded contexts, aggregates, value objects, invariants, events.
9	Session timeout in a blocking CLI was ambiguous.	Expiry check on every input return plus optional watchdog; injectable `Clock` for tests.

Contents

Document control 2

What changed in version 2.0 . . . 2

Contents 3

1. Purpose and business objective 6

2. Scope 6

3. Stakeholders and roles 6

4. Requirements 7

4.1 Functional requirements . . . 7

4.2 Non-functional requirements . . . 7

5. Assumptions and business rules 7

5.1 Account-type rules (simplified, fixed for this exercise) . . . 8

6. High-level architecture 8

6.1 Key architectural decisions . . . 8

6.2 Layer view . . . 9

7. Component view 10

8. Deployment view 10

9. Use cases 11

10. Security architecture (summary) 12

11. Data design overview 12

12. Transaction, concurrency and timeout strategy 12

13. Package structure 13

14. Class design 14

15. Service contracts 15

16. DAO contracts 16

17. Database design 16

17.1 Reference schema (MySQL 8+) . . . 18

17.2 PostgreSQL differences . . . 19

17.3 Business rule → constraint mapping . . . 19

18. Detailed algorithms 19

18.1 Registration . . . 19

18.2 Login . . . 20
18.3 Deposit and withdrawal . . . 20
18.4 Fund transfer . . . 21
18.5 Deposit account and loan repayment . . . 22
18.6 Transaction history and pagination . . . 22

19. State machines **23**

20. Session management **24**

21. Validation matrix **24**

22. Exception and error handling **25**

23. Configuration **25**

24. Logging and audit design **25**

25. Testing strategy **26**

Essential test cases . . . 26

26. Build and run guidance **27**

27. Implementation plan **27**

28. Approach and scope **28**

29. Ubiquitous language **28**

30. Bounded contexts and context map **29**

31. Aggregates, entities and value objects **30**

31.1 Tactical building blocks . . . 30
31.2 Aggregate design rules applied . . . 30

32. Domain services and policies **31**

33. Domain events **31**

34. Repositories and layering **32**

35. Business invariants catalogue **32**

36. Requirement-to-design traceability **33**

37. Definition of done and acceptance gates **33**

38. Evaluation rubric (100 marks) **33**

39. AI-assisted development and participant rotation **34**

40. Final submission checklist **34**

41. Limitations and future enhancements **34**

PART A

High-Level Design (HLD)

What the system is, how it is structured, and why

1. Purpose and business objective

Build a menu-driven Java terminal application in which a registered customer can securely log in, manage multiple bank-account records, perform eligible financial operations, review balances and transaction history, and log out or be logged out automatically after inactivity. The application must demonstrate genuine Core Java, JDBC and SQL competency **without** Spring Boot, React or any web framework, and it is the foundation for project P2.

Design principles

- **Database is the source of truth** for balances and the ledger; Java-side checks are never sufficient on their own.
- **Authorize on every call**: identity comes from the session, never from user-typed customer IDs.
- **Atomic money movement**: balance changes and the ledger row commit together or not at all.
- **Layered and testable**: presentation, service, DAO, with security and validation as cross-cutting concerns.
- **Simple but honest**: each account type has explicit, documented rules instead of a generic balance.

2. Scope

In scope	Out of scope
Customer registration and profile; username/password login with salted adaptive hashing; Savings, Current, Deposit and Loan accounts; deposit, withdrawal, transfer, loan repayment; balance enquiry; paginated history; inactivity timeout and logout; audit records; JDBC transactions; JUnit tests; documented clean-database setup.	Real money movement; payment-network integration; KYC/regulatory workflows; interest engines and loan origination/disbursement; production identity provider; MFA; internet-facing APIs; distributed services; fraud detection; GUI/web UI.

3. Stakeholders and roles

Role	Responsibilities / permissions
Customer	Register, log in, view own profile, create account records, transact on owned eligible accounts, view own history, change password, log out.
Application	Validates input, enforces authorization, hashes/verifies passwords, manages session expiry, coordinates JDBC transactions, writes ledger and audit.
Database	Enforces PK/FK/UQ/CHECK constraints, persists state, provides isolation, locking and atomic commit.
Trainer / evaluator	Reviews code and schema, runs tests, observes security and rollback demonstrations. No customer-facing administrative privilege.

4. Requirements

4.1 Functional requirements

ID	Requirement	Brief	Acceptance criteria
F01	Registration	P1-01	Unique username and email; validated fields; password policy; only a hash is stored.
F02	Authentication	P1-02	Valid credentials create a session; invalid ones are denied with a generic message and audited.
F03	Customer profile	P1-03	View profile; change password (re-authenticate with current password). Permitted fields only.
F04	Account creation	P1-04	Create Savings, Current, Deposit, Loan records; unique generated account number; owner linked.
F05	Multiple accounts	P1-05	One customer owns many accounts and can list only their own.
F06	Deposit	P1-06	Positive amount to an owned, active, eligible account; balance and ledger consistent.
F07	Withdrawal	P1-07	Only when sufficient available balance exists after locking and re-read.
F08	Fund transfer	P1-08	Debit, credit and ledger in one transaction; rollback on any failure.
F09	Balance enquiry	P1-09	Account details and balance for owned accounts only.
F10	History	P1-10	Dated, reference-numbered records with status; stable pagination.
F11	Session management	P1-11	Idle timeout, invalidation and explicit logout.
F12	Database integrity	P1-12	Keys, constraints, prepared statements, DECIMAL money, transactional consistency.
F13	Audit	P1-13	Login outcomes and financial events with reference, timestamp, status.
F14	Testing	P1-14	Success, invalid input, insufficient funds, authorization, rollback and concurrency cases automated.

4.2 Non-functional requirements

Attribute	Design target
Security	Adaptive salted hashing; no plaintext or reversible encryption; parameterized SQL; authorization at the service boundary; secrets from environment variables.
Integrity	BigDecimal in Java and DECIMAL(19,2) in SQL; explicit scale/rounding policy; one DB transaction per transfer; row locking.
Maintainability	Packages: app, model, service, dao, security, validation, exception, util; no SQL outside dao; no business rules in app.
Reliability	try-with-resources; explicit rollback; typed exceptions; safe handling of connection loss.
Testability	Service logic testable with fake DAOs and an injectable Clock; DAO tests against a dedicated test schema.
Usability	Numbered menus, robust invalid-input and EOF handling, user-safe messages.
Portability	Standard JDBC; documented MySQL vs PostgreSQL schema differences.
Observability	Structured logs with no passwords, hashes, DB credentials or full account data.

5. Assumptions and business rules

- A customer has one unique ID and may own many accounts; an account has exactly one owner.
- Account numbers and transaction references are unique and produced by a controlled generator (`ReferenceGenerator`).
- Money: `BigDecimal`, scale 2, `RoundingMode.UNNECESSARY` (inputs with more than two decimals are rejected, not rounded). Configurable per-operation limits (default maximum 1,000,000.00).
- Overdraft is **not** allowed in the baseline: Current accounts use overdraft limit 0. Enabling an overdraft is a documented extension.
- A session timeout is distinct from a database transaction timeout (see §12).

5.1 Account-type rules (simplified, fixed for this exercise)

Type	Balance meaning	Deposit	Withdraw	Transfer	Other rules
SAVINGS	Spendable cash	Yes	Yes (funds ≥ amount)	Source and destination	Minimum balance 0 (configurable).
CURRENT	Spendable cash	Yes	Yes (funds ≥ amount)	Source and destination	Overdraft limit 0 in baseline.
DEPOSIT	Locked principal	Only the opening deposit (funded from an owned Savings/Current account)	Only when MATURED; proceeds go to an owned Savings/Current account and the account closes	Not a normal transfer endpoint	Term in months; maturity_date fixed at creation; early closure refused (penalty rules are a future enhancement).
LOAN	Outstanding principal (amount owed)	No	Never (not spendable cash)	Not a normal transfer endpoint	Created with a principal by a documented simplified flow (no disbursement). Repayment is a debit from an owned Savings/Current account that reduces the balance; overpayment refused; zero balance closes the loan.

Scope guidance. Implement Savings and Current first (milestones M3–M5), then add the simplified Deposit and Loan workflows. All four types must exist, be validated and be tested before final evaluation.

6. High-level architecture

The application is a single-process, layered Java application. The presentation layer parses input and renders menus. Services own use cases, authorization, business rules and transaction boundaries. DAOs isolate SQL and JDBC. Security, validation, exceptions and utilities are cross-cutting. The relational database remains the source of truth.

6.1 Key architectural decisions

ID	Decision	Rationale / consequence
AD-1	Plain Java + JDBC, no framework.	Required by the brief; forces understanding of transactions, SQL and resource handling.
AD-2	Layered architecture with DAO interfaces.	Separation of concerns; services testable with fakes.
AD-3	Service layer owns the transaction boundary; DAOs receive a `Connection`.	A multi-step operation must share one connection; DAOs must not commit on their own.
AD-4	Pessimistic row locking: `SELECT ... FOR UPDATE` in ascending `account_id` order.	Prevents lost updates and overdrawing; deterministic order avoids most deadlocks.
AD-5	Isolation READ COMMITTED set explicitly; balances re-read after locking.	MySQL InnoDB defaults to REPEATABLE READ and PostgreSQL to READ COMMITTED; an explicit level avoids dialect surprises.
AD-6	Single ledger row per business transaction.	Simple to explain and test; history for an account is a UNION of its source and destination rows; balance-after snapshots support reconciliation.
AD-7	Separate audit tables; failures never appear as SUCCESS ledger rows.	Ledger stays a record of completed money movement; audit records attempts.
AD-8	Password hashing through a `PasswordHasher` interface (bcrypt cost 12 default; Argon2id or PBKDF2WithHmacSHA256 acceptable).	Swappable algorithm; verify-only API; never log inputs.
AD-9	In-memory session with injectable `Clock`.	No persistent sessions needed in a single-process CLI; deterministic timeout tests.
AD-10	Configuration: `app.properties` for non-secrets, environment variables for credentials.	No secrets in Git.

6.2 Layer view

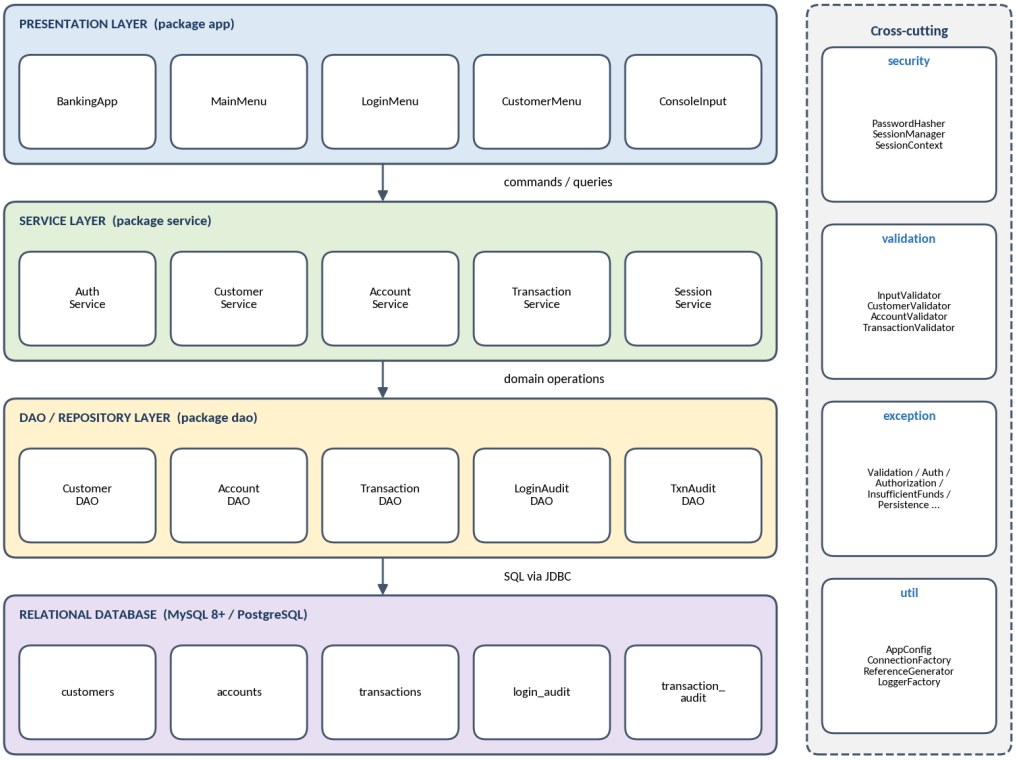


Figure 1. Layered architecture and cross-cutting components

7. Component view

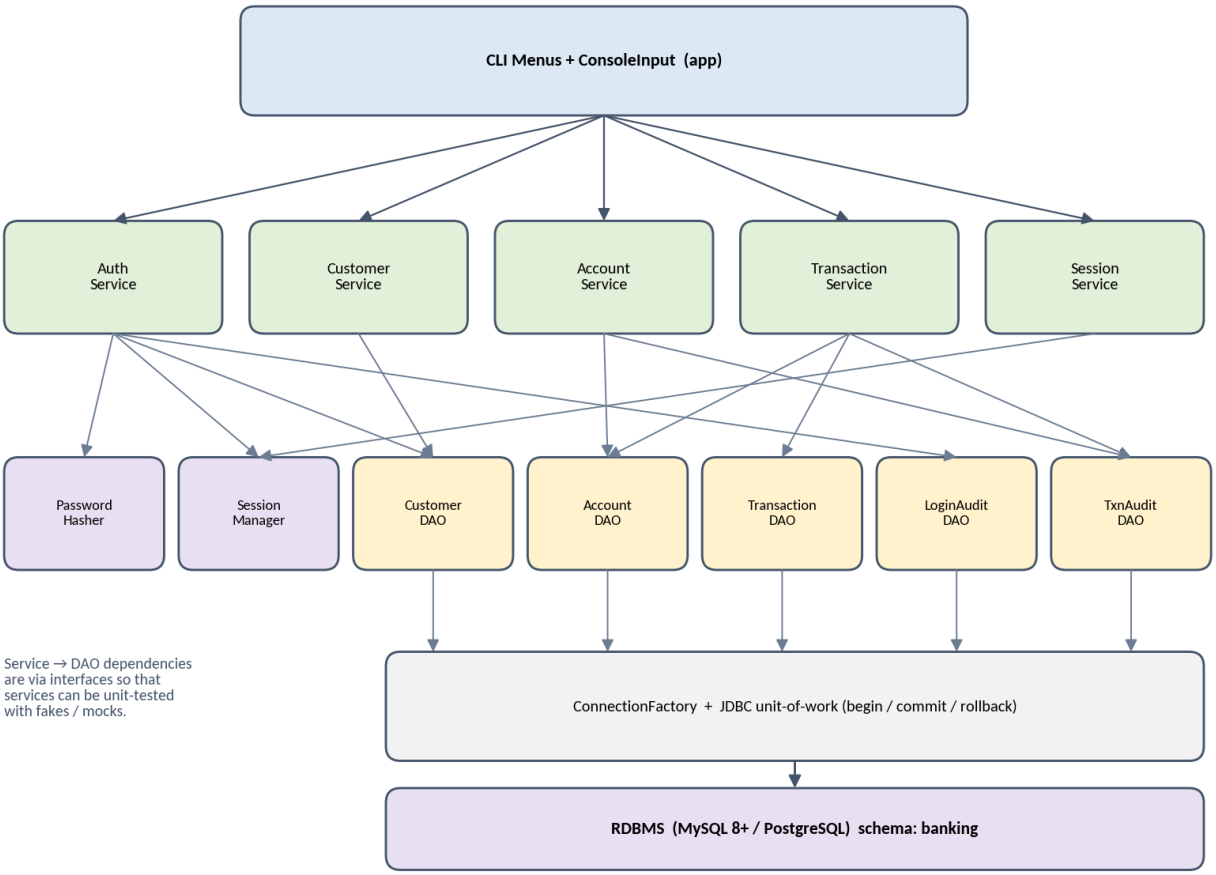


Figure 2. Logical components and dependencies

8. Deployment view

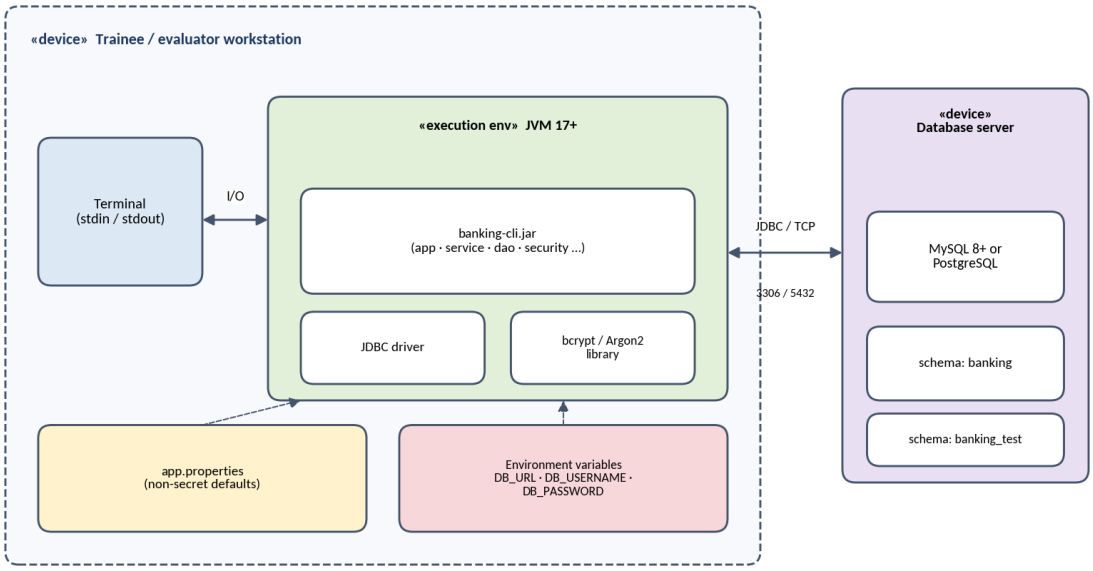


Figure 3. Deployment (single workstation, local database)

Use a local database for training, a least-privilege database user (DML on the banking schema only), credentials in environment variables, and a separate `banking_test` schema for automated tests. The diagram is logical and does not imply production hardening.

9. Use cases

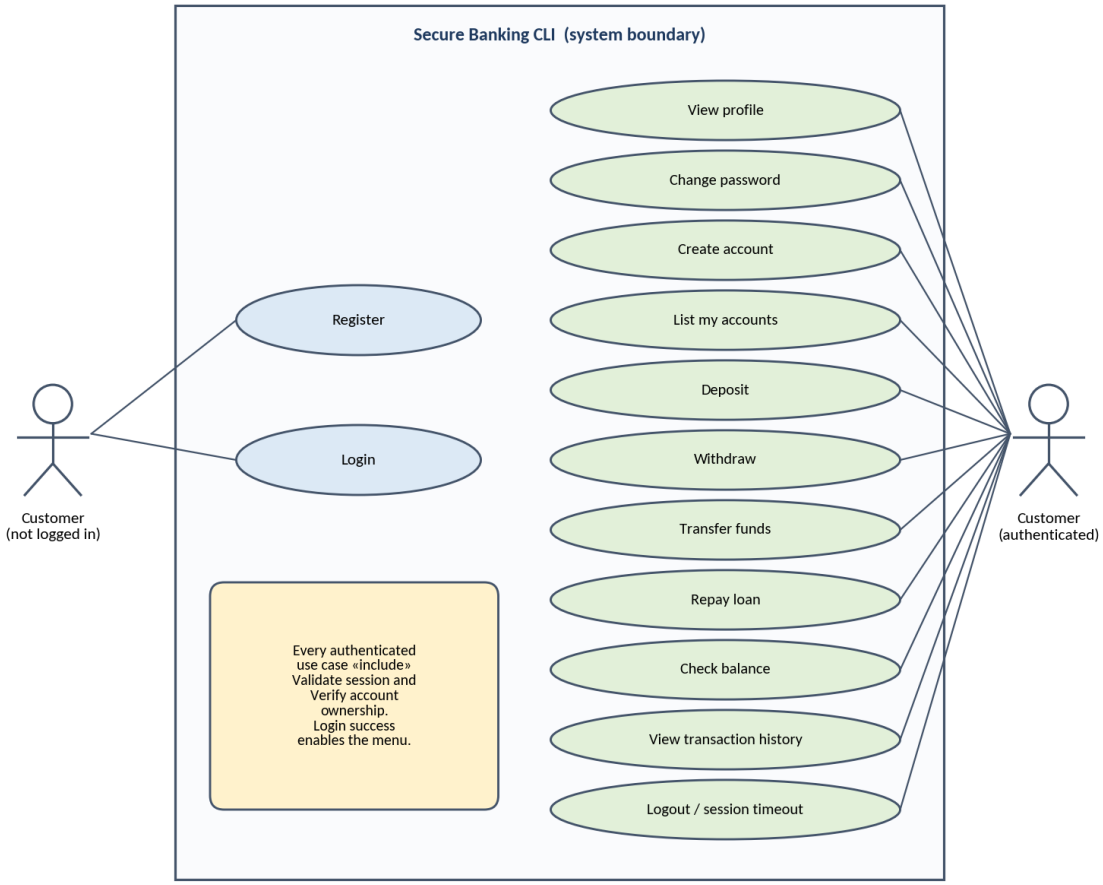


Figure 4. Customer use cases

Use case	Actor	Main flow (summary)	Key alternate / failure flows
Register	Anonymous	Enter details → validate → hash password → save customer.	Duplicate username/email; weak password; invalid format.
Login	Anonymous	Enter credentials → verify → create session → show menu.	Unknown user / wrong password (generic error, audited); locked account.
Create account	Authenticated	Choose type → validate → generate number → save.	Invalid type; missing deposit term / loan principal.
Deposit / Withdraw	Authenticated	Choose owned account → amount → transaction → receipt.	Not owner; frozen/closed; insufficient funds; bad amount.
Transfer	Authenticated	Source, destination, amount → locked atomic transfer → receipt.	Ineligible destination; same account; insufficient funds; DB failure → rollback.
Repay loan	Authenticated	Choose loan + funding account → debit funding, reduce loan.	Overpayment; loan closed; funding not owned.
History	Authenticated	Choose account → filter → page through results.	Page out of range; account not owned.
Logout / timeout	Authenticated / system	Clear session; audit; return to welcome.	Idle timeout detected on next input.

10. Security architecture (summary)

Control	Design
Password storage	Salted adaptive hash (bcrypt/Argon2id/PBKDF2) with parameters encoded in the stored value. Plain SHA-256 or reversible encryption are not acceptable.
Login	Constant-effort verification (dummy hash when the user does not exist); generic failure message; failed-attempt counter with temporary lockout.
Authorization	Every service method receives the authenticated `customerId` from the session and verifies account ownership; DAO queries include `customer_id` where relevant.
SQL injection	PreparedStatement for every user-supplied value; whitelist any dynamic ORDER BY / column name.
Session	Idle timeout checked before each protected operation; logout clears state; no customer ID taken from CLI input.
Error handling	Typed exceptions mapped to safe messages; stack traces and SQL only in restricted logs.
Audit	Login outcomes and financial event references/timestamps/status; never passwords, hashes or credentials.

11. Data design overview

Four core tables from the brief (`customers, accounts, transactions, login_audit`) plus three supporting tables: `transaction_audit` (financial events including failures), `deposit_details` and `loan_details` (type-specific attributes). The full ERD and DDL are in Part B, §17.

12. Transaction, concurrency and timeout strategy

- **One connection per unit of work.** Disable auto-commit, perform all steps, commit once, rollback on any failure, restore auto-commit and close in `finally`/try-with-resources.
- **Locking.** Lock all involved account rows with `SELECT ... FOR UPDATE` ordered by `account_id`; re-read balances after the lock.
- **Deadlocks and lock waits.** Retry the whole unit of work at most 3 times with short randomized backoff on MySQL error 1213 / SQLState 40001 or PostgreSQL SQLState 40P01; fail with a typed exception afterwards.
- **Idempotency.** Unique transaction reference prevents duplicate application if a request is retried.
- **Session timeout** (application, default 5 minutes idle) is unrelated to the **database transaction timeout** (statement/lock wait limits, default 10 s via `Statement.setQueryTimeout` and DB lock-wait settings).

PART B

Low-Level Design (LLD)

Packages, classes, contracts, schema, algorithms and tests

13. Package structure

```
banking-cli/
├── pom.xml
├── README.md                    setup, configuration, run, AI-assistance log
├── sql/
│   ├── mysql/schema.sql         reference DDL (MySQL 8+)
│   ├── postgres/schema.sql      PostgreSQL variant
│   └── sample-data.sql          fake data only; no reusable passwords
├── src/main/resources/app.properties
├── src/main/java/com/example/banking/
│   ├── app/          BankingApplication, MainMenu, LoginMenu, CustomerMenu, ConsoleInput
│   ├── model/        Customer, Account (abstract), SavingsAccount, CurrentAccount, DepositAccount, LoanAccount,
│   │                 BankTransaction, LoginAudit, TransactionAudit, Money, AccountNumber,
│   │                 AccountType, AccountStatus, CustomerStatus, TransactionType, TransactionStatus
│   ├── service/      AuthService, CustomerService, AccountService, TransactionService, SessionService
│   ├── dao/          CustomerDAO, AccountDAO, TransactionDAO, LoginAuditDAO, TransactionAuditDAO  (+ jdbc/ impls)
│   ├── security/     PasswordHasher, BcryptPasswordHasher, SessionContext, SessionManager
│   ├── validation/   InputValidator, CustomerValidator, AccountValidator, TransactionValidator
│   ├── exception/    BankingException (base), ValidationException, AuthenticationException,
│   │                 AuthorizationException, InsufficientFundsException, AccountNotEligibleException,
│   │                 PersistenceException, SessionExpiredException
│   └── util/         AppConfig, ConnectionFactory, JdbcTransaction, ReferenceGenerator, LoggerFactory
└── src/test/java/com/example/banking/   unit, service, DAO-integration and transaction tests
```

Package	Responsibility	Must not
app	Startup, menu routing, input parsing and rendering.	Contain SQL or financial rules.
model	Domain state, enums, value objects, invariants.	Know about JDBC or the console.
service	Use cases, authorization, rules, transaction boundaries.	Build SQL strings; read from the console.
dao	Parameterized SQL and ResultSet mapping.	Commit/rollback on their own; apply business rules.
security	Hashing, session lifecycle.	Log secrets.
validation	Syntactic and business-input validation.	Access the database.
exception	Typed, user-safe failures.	Leak SQL or internals in messages.
util	Config, connections, references, logging.	Hold business logic.

14. Class design

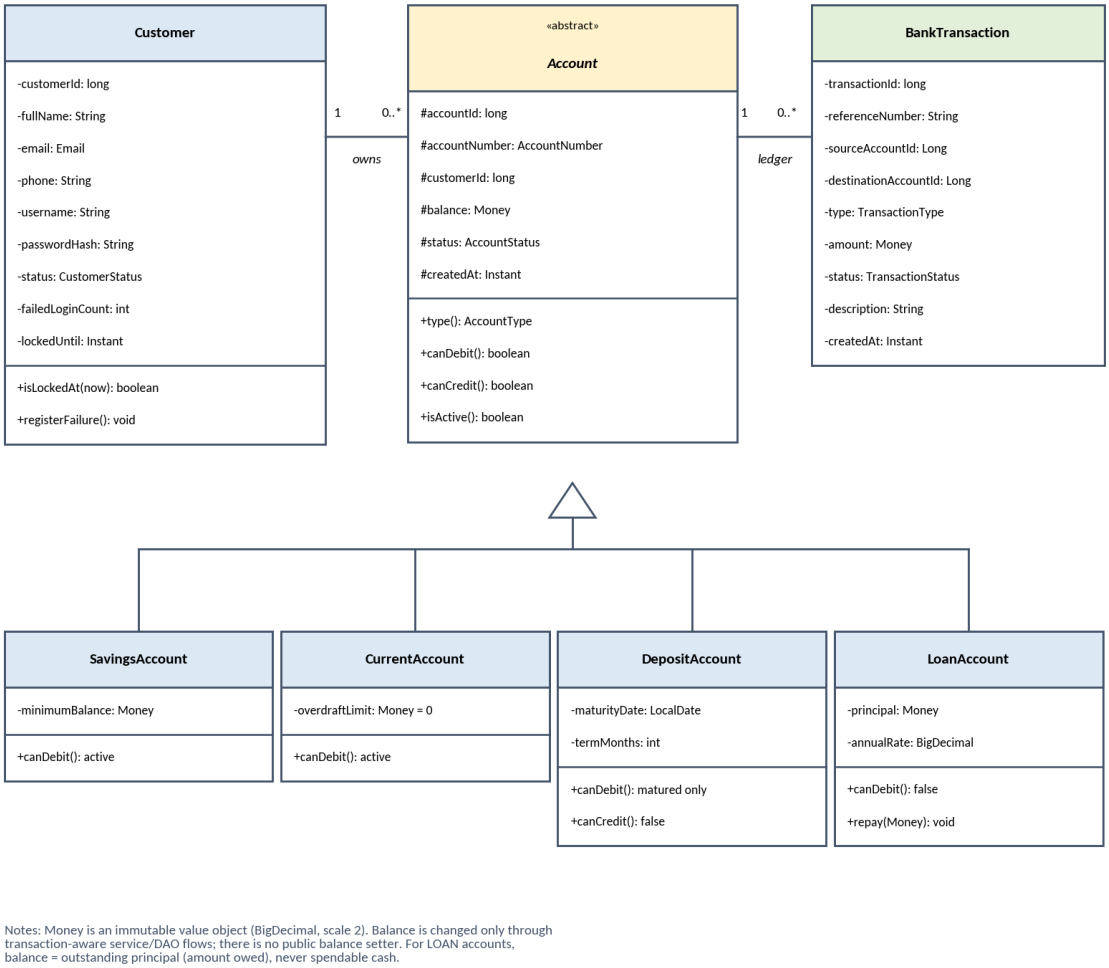


Figure 5. Domain class model (simplified)

14.1 Design notes

- **Account** is an abstract class holding shared state and invariants. Subclasses exist only where behaviour truly differs (`canDebit()`, `canCredit()`, type-specific fields). Do not add inheritance merely to satisfy a checklist.
- **No public balance setter.** Balance changes occur only inside service/DAO flows that hold the row lock.
- **Money** is an immutable value object wrapping `BigDecimal` (scale 2); methods `add, subtract, isPositive, compareTo`; constructor rejects excess scale and nulls.
- **BankTransaction** is immutable after creation: reference, type, amount, optional source/destination IDs, status, description, timestamp.
- **PasswordHasher**: `String hash(char[] raw)`, `boolean verify(char[] raw, String encoded)`. Never logs inputs; callers zero the `char[]` after use.
- **DAO interfaces** let services be tested with in-memory fakes; JDBC implementations live in `dao.jdbc`.

14.2 Service ↔ DAO / collaborator dependencies

Service	Depends on	Responsibility
AuthService	CustomerDAO, LoginAuditDAO, PasswordHasher, SessionManager, CustomerValidator	Register, authenticate, lockout, change password.
CustomerService	CustomerDAO, CustomerValidator	View/update permitted profile fields.
AccountService	AccountDAO, CustomerDAO, AccountValidator, ReferenceGenerator	Open accounts, list owned accounts, enforce eligibility and status rules.
TransactionService	AccountDAO, TransactionDAO, TransactionAuditDAO, JdbcTransaction, TransactionValidator, ReferenceGenerator, Clock	Deposit, withdraw, transfer, loan repayment, history; owns the DB transaction boundary.
SessionService	SessionManager, Clock	Expiry checks, activity refresh, invalidation.

15. Service contracts

Method (illustrative)	Preconditions	Postconditions / failures
AuthService.register(RegisterRequest)	Valid fields; unique username/email; password meets policy.	Customer saved with encoded hash; no raw password persisted. ValidationException on bad input or duplicates.
AuthService.authenticate(username, char[] pwd)	Non-blank input; DB available.	SessionContext on success; on failure audited and AuthenticationException (generic). Lockout after N failures.
AuthService.changePassword(session, current, new)	Valid session; current password verifies; new password meets policy.	Hash replaced; audit event; session retained.
AccountService.openAccount(customerId, type, params)	Authenticated; valid type; type-specific parameters supplied.	Unique account number; owner linked; initial balance per type policy.
AccountService.getOwnedAccounts(customerId)	customerId from session.	Only that customer's accounts.
TransactionService.deposit(customerId, accountId, amount)	Positive amount; owned, ACTIVE, Savings/Current.	Balance increased and DEPOSIT ledger row inserted in one commit.
TransactionService.withdraw(customerId, accountId, amount)	Positive amount; owned, ACTIVE; funds sufficient after lock.	Balance decreased and WITHDRAWAL row inserted atomically; InsufficientFundsException otherwise.
TransactionService.transfer(customerId, srcId, dstNo, amount)	Owned source; destination exists, ACTIVE, Savings/Current; distinct accounts.	Debit + credit + TRANSFER row in one commit or full rollback.
TransactionService.repayLoan(customerId, fundingId, loanId, amount)	Both owned; funding Savings/Current; loan ACTIVE; amount ≤ outstanding.	Funding debited, loan reduced, LOAN_REPAYMENT row; loan closed at zero.
TransactionService.history(customerId, accountId, filter, page)	Owned account; page ≥ 0; size ≤ 50.	Stable ordered page with direction (credit/debit) per row.
SessionService.isExpired(session, now)	Session has lastActivity; idle duration configured.	True session invalidated, TIMEOUT audited.

16. DAO contracts

DAO	Key operations (all PreparedStatement)
CustomerDAO	insert(Customer); findById; findByUsername; existsByUsername/Email; updateProfile; updatePasswordHash; recordFailedLogin/reset; setLockedUntil.
AccountDAO	insert(Account); findOwnedById(accountId, customerId); findByNumber; listByCustomer; lockForUpdate(conn, ids[] ascending); updateBalance(conn, id, newBalance); updateStatus.
TransactionDAO	insert(conn, BankTransaction); pageByAccount(accountId, filter, limit, offset); findByReference.
LoginAuditDAO	record(customerId?, usernameAttempted, outcome) — append-only; own short transaction.
TransactionAuditDAO	record(customerId, reference, eventType, outcome, failureCode) — append-only; written outside the money transaction so that failures survive a rollback.

17. Database design

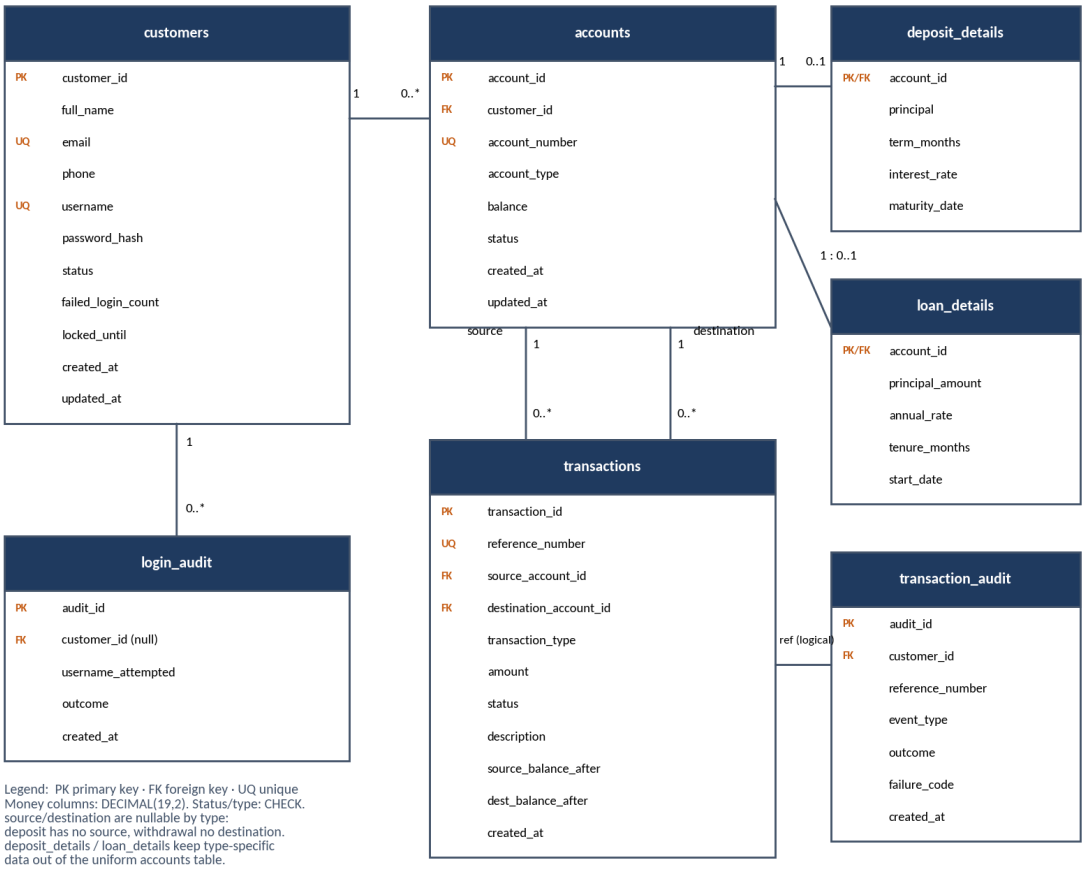


Figure 6. Entity-relationship diagram

Relationship	Cardinality	Rule
Customer → Account	1 : 0..*	Each account has exactly one owner; a customer may own many accounts.
Account → Transaction (source)	1 : 0..*	Source is null for deposits.
Account → Transaction (destination)	1 : 0..*	Destination is null for withdrawals; required for transfers and repayments.
Account → deposit_details / loan_details	1 : 0..1	Only for DEPOSIT / LOAN accounts respectively.
Customer → login_audit	1 : 0..*	customer_id null when the attempted username is unknown.

Relationship	Cardinality	Rule
Transaction ↔ transaction_audit	1 : 0..*	Logical link by reference number; failed attempts have no ledger row.

17.1 Reference schema (MySQL 8+)

```
CREATE TABLE customers (
  customer_id        BIGINT PRIMARY KEY AUTO_INCREMENT,
  full_name          VARCHAR(120) NOT NULL,
  email              VARCHAR(254) NOT NULL UNIQUE,
  phone              VARCHAR(30),
  username           VARCHAR(80)  NOT NULL UNIQUE,
  password_hash      VARCHAR(255) NOT NULL,
  status             VARCHAR(20)  NOT NULL DEFAULT 'ACTIVE',
  failed_login_count INT          NOT NULL DEFAULT 0,
  locked_until       TIMESTAMP    NULL,
  created_at         TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at         TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT chk_customer_status CHECK (status IN ('ACTIVE','LOCKED','CLOSED')),
  CONSTRAINT chk_failed_count    CHECK (failed_login_count >= 0)
);

CREATE TABLE accounts (
  account_id     BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_id    BIGINT        NOT NULL,
  account_number VARCHAR(34)   NOT NULL UNIQUE,
  account_type   VARCHAR(20)   NOT NULL,
  balance        DECIMAL(19,2) NOT NULL DEFAULT 0.00,   -- LOAN: outstanding principal
  status         VARCHAR(20)   NOT NULL DEFAULT 'ACTIVE',
  created_at     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_account_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  CONSTRAINT chk_account_type    CHECK (account_type IN ('SAVINGS','CURRENT','DEPOSIT','LOAN')),
  CONSTRAINT chk_account_status  CHECK (status IN ('ACTIVE','FROZEN','MATURED','CLOSED')),
  CONSTRAINT chk_account_balance CHECK (balance >= 0),
  INDEX idx_accounts_customer (customer_id)
);

CREATE TABLE deposit_details (
  account_id    BIGINT PRIMARY KEY,
  principal     DECIMAL(19,2) NOT NULL,
  term_months   INT           NOT NULL,
  interest_rate DECIMAL(5,2)  NOT NULL DEFAULT 0.00,
  maturity_date DATE          NOT NULL,
  CONSTRAINT fk_dep_account FOREIGN KEY (account_id) REFERENCES accounts(account_id),
  CONSTRAINT chk_dep_term   CHECK (term_months > 0 AND principal > 0)
);

CREATE TABLE loan_details (
  account_id       BIGINT PRIMARY KEY,
  principal_amount DECIMAL(19,2) NOT NULL,
  annual_rate      DECIMAL(5,2)  NOT NULL DEFAULT 0.00,
  tenure_months    INT           NOT NULL,
  start_date       DATE          NOT NULL,
  CONSTRAINT fk_loan_account FOREIGN KEY (account_id) REFERENCES accounts(account_id),
  CONSTRAINT chk_loan_vals   CHECK (principal_amount > 0 AND tenure_months > 0)
);

CREATE TABLE transactions (
  transaction_id         BIGINT PRIMARY KEY AUTO_INCREMENT,
  reference_number       VARCHAR(40)   NOT NULL UNIQUE,
  source_account_id      BIGINT        NULL,
  destination_account_id BIGINT        NULL,
  transaction_type       VARCHAR(20)   NOT NULL,
  amount                 DECIMAL(19,2) NOT NULL,
  status                 VARCHAR(20)   NOT NULL,
  description            VARCHAR(255),
  source_balance_after   DECIMAL(19,2) NULL,
  dest_balance_after     DECIMAL(19,2) NULL,
  created_at             TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_txn_source FOREIGN KEY (source_account_id)      REFERENCES accounts(account_id),
  CONSTRAINT fk_txn_dest   FOREIGN KEY (destination_account_id) REFERENCES accounts(account_id),
  CONSTRAINT chk_txn_amount CHECK (amount > 0),
  CONSTRAINT chk_txn_type   CHECK (transaction_type IN ('DEPOSIT','WITHDRAWAL','TRANSFER','LOAN_REPAYMENT')),
  CONSTRAINT chk_txn_status CHECK (status IN ('PENDING','SUCCESS','FAILED','REVERSED')),
  CONSTRAINT chk_txn_shape  CHECK (
       (transaction_type = 'DEPOSIT'    AND source_account_id IS NULL     AND destination_account_id IS NOT NULL)
    OR (transaction_type = 'WITHDRAWAL' AND source_account_id IS NOT NULL AND destination_account_id IS NULL)
    OR (transaction_type IN ('TRANSFER','LOAN_REPAYMENT')
        AND source_account_id IS NOT NULL AND destination_account_id IS NOT NULL
        AND source_account_id <> destination_account_id)),
  INDEX idx_txn_src_time (source_account_id, created_at, transaction_id),
  INDEX idx_txn_dst_time (destination_account_id, created_at, transaction_id)
```

```
);

CREATE TABLE login_audit (
  audit_id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_id        BIGINT      NULL,
  username_attempted VARCHAR(80) NOT NULL,
  outcome            VARCHAR(20) NOT NULL,
  created_at         TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_audit_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  CONSTRAINT chk_audit_outcome CHECK (outcome IN ('SUCCESS','FAILURE','LOCKED','TIMEOUT','LOGOUT'))
);

CREATE TABLE transaction_audit (
  audit_id         BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_id      BIGINT      NOT NULL,
  reference_number VARCHAR(40) NULL,
  event_type       VARCHAR(30) NOT NULL,   -- DEPOSIT, WITHDRAWAL, TRANSFER, LOAN_REPAYMENT, ...
  outcome          VARCHAR(20) NOT NULL,   -- SUCCESS, REJECTED, FAILED, ROLLED_BACK
  failure_code     VARCHAR(40) NULL,       -- e.g. INSUFFICIENT_FUNDS, NOT_OWNER; never raw SQL/stack trace
  created_at       TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_tauditcust FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  INDEX idx_taudit_customer (customer_id, created_at)
);
```

17.2 PostgreSQL differences

Topic	MySQL 8+	PostgreSQL
Auto keys	`BIGINT AUTO_INCREMENT`	`BIGINT GENERATED ALWAYS AS IDENTITY`
Timestamps	`TIMESTAMP … ON UPDATE CURRENT_TIMESTAMP`	`TIMESTAMPTZ`; use a trigger or set `updated_at` in the DAO.
Indexes	Inline `INDEX` in `CREATE TABLE`	Separate `CREATE INDEX` statements.
Row lock	`SELECT … FOR UPDATE` (InnoDB)	`SELECT … FOR UPDATE`
Deadlock signal	Error 1213 / SQLState 40001	SQLState 40P01
CHECK enforcement	Enforced from 8.0.16; verify the version	Always enforced

17.3 Business rule → constraint mapping

Rule	Enforced by
Unique username/email/account number/reference	UNIQUE constraints (plus pre-checks for friendly messages).
No negative balance; positive amounts; valid status/type	CHECK constraints (and service validation before the statement).
Shape of each ledger row per type	`chk_txn_shape`.
Ownership of accounts and rows	FKs plus `WHERE account_id=? AND customer_id=?` in owner-scoped DAO methods.
Eligibility (type/status), maturity, repayment limits	Service/domain layer (cannot be expressed fully in portable CHECKs).

18. Detailed algorithms

18.1 Registration

- Validate name, email format, phone, username pattern (3–30 chars, letters/digits/._-), password policy (min 10 chars, mixed classes, not equal to username).
- Check uniqueness (friendly message) and rely on UNIQUE constraints for races; map `SQLIntegrityConstraintViolation` to `ValidationException`.
- Hash the password; insert customer with status ACTIVE; zero the `char[]`.

18.2 Login

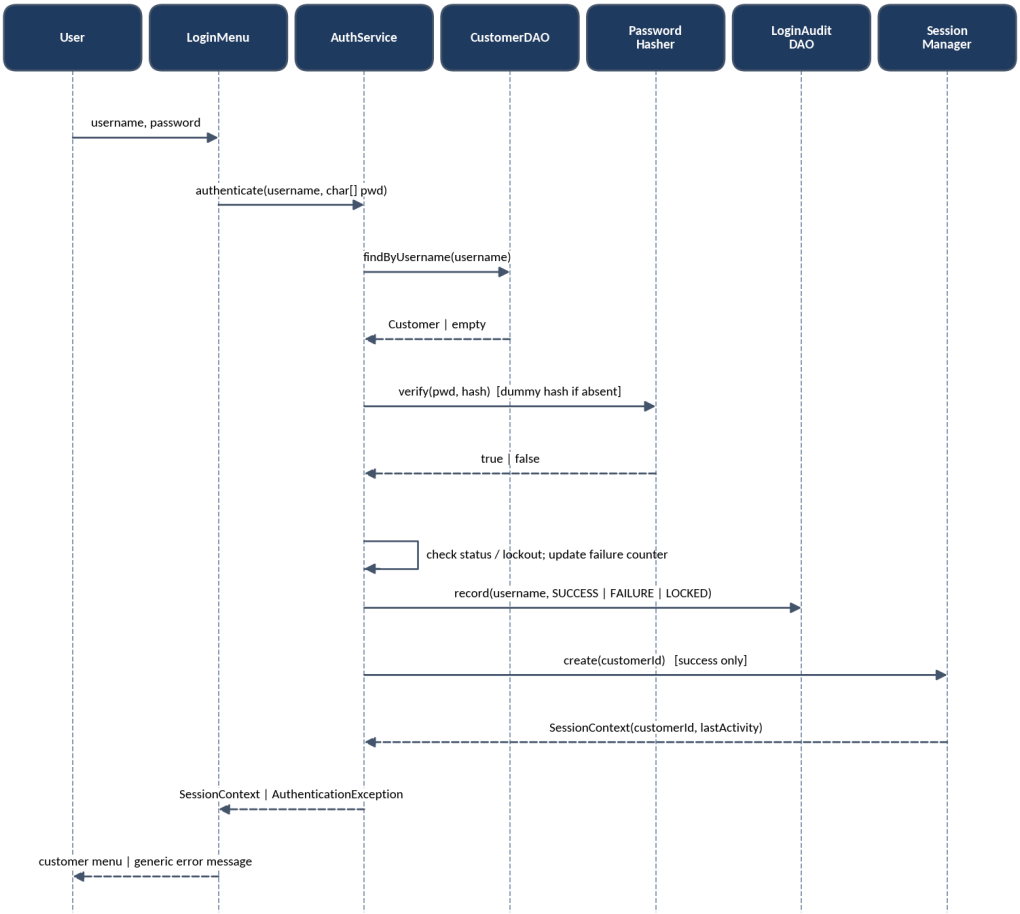


Figure 7. Login sequence (success and failure paths)

- Read credentials without logging; validate non-blank.
- Fetch customer by username (PreparedStatement). If absent, still verify against a fixed dummy hash to equalize timing; then audit FAILURE and return a generic error.
- If status is LOCKED/CLOSED or `locked_until` is in the future, audit LOCKED and return the same generic message.
- Verify the hash. On failure increment `failed_login_count`; at 5 failures set `locked_until = now + 15 min`. On success reset the counter.
- On success create `SessionContext(customerId, lastActivity)`, audit SUCCESS, show the customer menu.

18.3 Deposit and withdrawal

```
deposit(customerId, accountId, amount):
    validator.requirePositiveMoney(amount)
    return tx.execute(conn -> {                          // begin; auto-commit off
        Account a = accountDao.lockOwned(conn, accountId, customerId);   // FOR UPDATE, ownership in WHERE
        rules.requireCanCredit(a);                        // ACTIVE and SAVINGS/CURRENT
        Money after = a.balance().add(amount);
        accountDao.updateBalance(conn, a.id(), after);
        txnDao.insert(conn, BankTransaction.deposit(ref(), a.id(), amount, after, SUCCESS));
        return receipt;
    });                                                  // commit, or rollback + rethrow

withdraw(...): same shape, but after the lock:
    rules.requireCanDebit(a);
    if (a.balance().compareTo(amount) < 0) throw new InsufficientFundsException();
```

18.4 Fund transfer

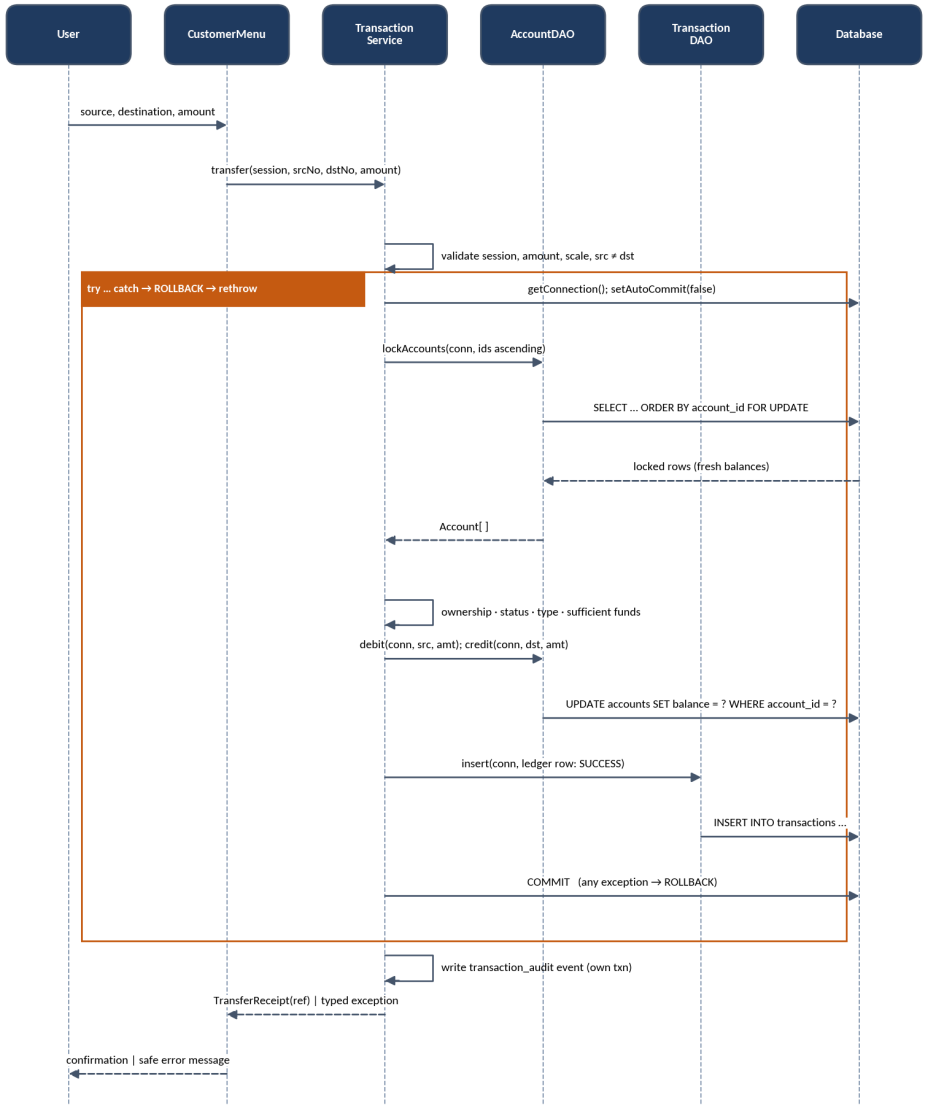


Figure 8. Transfer sequence and transaction boundary

```
transfer(customerId, srcId, dstNumber, amount):
    validator.requirePositiveMoney(amount)
    for attempt in 1..3:
      try:
        return tx.execute(conn -> {
            Account dst0 = accountDao.findByNumber(conn, dstNumber)      // resolve id (no lock yet)
                            .orElseThrow(AccountNotEligibleException::new);
            List<Account> locked = accountDao.lockForUpdate(conn, sorted(srcId, dst0.id()));  // ascending id
            Account src = pick(locked, srcId), dst = pick(locked, dst0.id());
            if (src.customerId() != customerId)            throw new AuthorizationException();
            rules.requireTransferable(src, dst);            // ACTIVE, SAVINGS/CURRENT, src != dst
            if (src.balance().compareTo(amount) < 0)       throw new InsufficientFundsException();
            Money sAfter = src.balance().subtract(amount), dAfter = dst.balance().add(amount);
            accountDao.updateBalance(conn, src.id(), sAfter);
            accountDao.updateBalance(conn, dst.id(), dAfter);
            txnDao.insert(conn, BankTransaction.transfer(ref(), src.id(), dst.id(), amount, sAfter, dAfter, SUCCESS));
            return receipt;
        });
      } catch (TransientDbException e) {                    // deadlock / lock timeout only
        if (attempt == 3) throw new PersistenceException("Please retry", e);
        sleep(randomBackoff());
      }
    // audit written after commit or after rollback, in its own short transaction
```

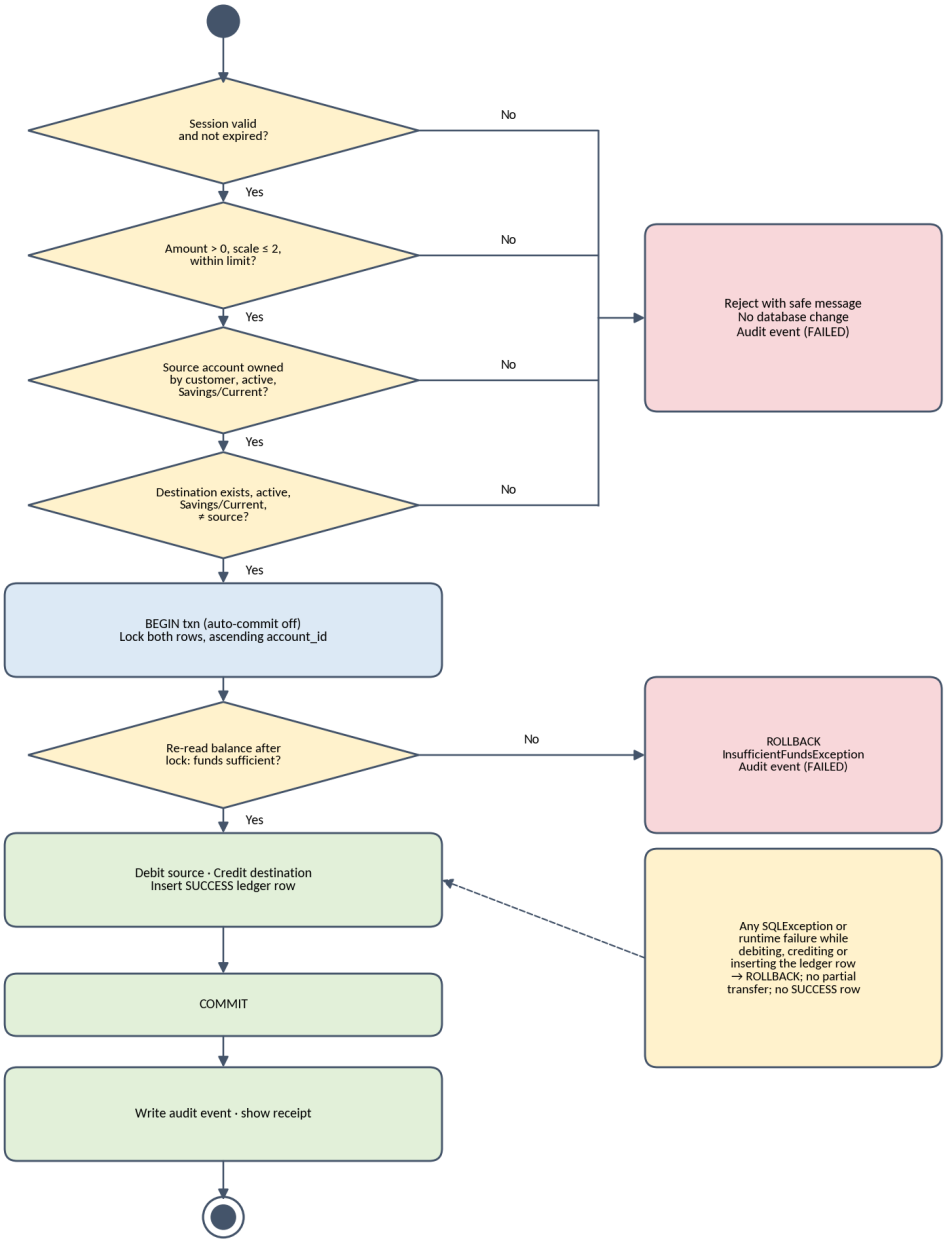


Figure 9. Transfer decision flow

18.5 Deposit account and loan repayment

- **Open deposit:** amount ≥ minimum; funding from an owned Savings/Current account via the same atomic pattern; insert account (balance = principal), `deposit_details` and a TRANSFER ledger row in one transaction.
- **Maturity:** on login/listing, the service marks ACTIVE deposits whose `maturity_date ≤ today` as MATURED (lazy evaluation; no scheduler needed).
- **Settle:** only MATURED; move principal to an owned Savings/Current account (TRANSFER row), set balance 0, status CLOSED.
- **Repay loan:** lock funding and loan rows in ascending ID order; require `amount ≤ loan.balance`; debit funding, reduce loan balance, insert LOAN_REPAYMENT row; close loan at zero.

18.6 Transaction history and pagination

```
SELECT t.transaction_id, t.reference_number, t.transaction_type, t.amount, t.status, t.created_at,
       CASE WHEN t.destination_account_id = ? THEN 'CREDIT' ELSE 'DEBIT' END AS direction
FROM   transactions t
WHERE  (t.source_account_id = ? OR t.destination_account_id = ?)
  AND  t.created_at >= ? AND t.created_at < ?          -- optional date filter
  AND  (? IS NULL OR t.transaction_type = ?)           -- optional type filter (whitelisted enum)
ORDER  BY t.created_at DESC, t.transaction_id DESC     -- stable ordering
LIMIT  ? OFFSET ?                                      -- page size 1..50; page >= 0
```

The account must first be verified as owned by the session customer; the page size is clamped, and no sort column is taken from user input.

19. State machines

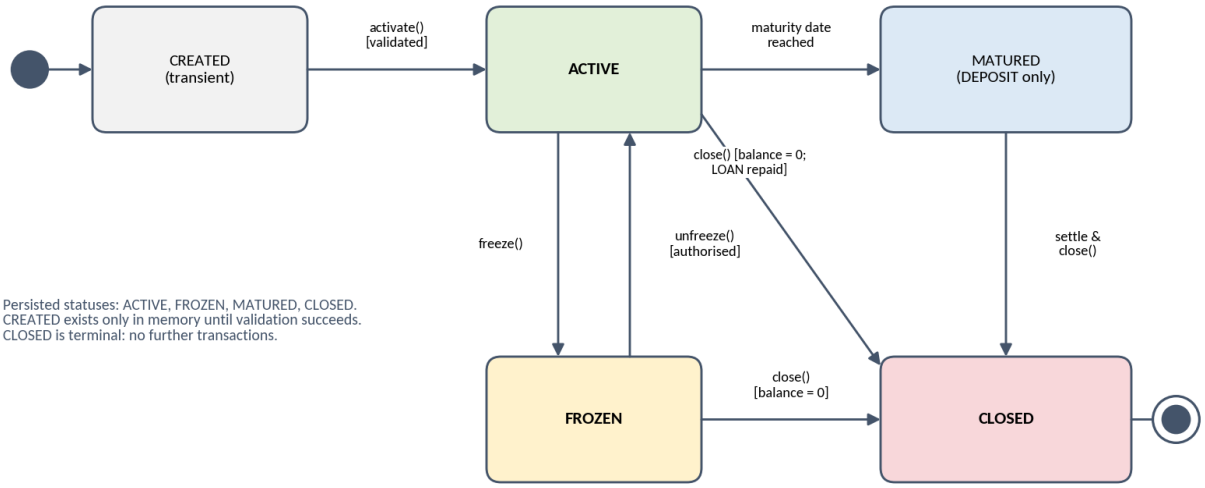


Figure 10. Account lifecycle

State	Permitted behaviour
CREATED	In-memory only; becomes ACTIVE when validation and persistence succeed.
ACTIVE	Operations allowed per account-type rules and ownership.
FROZEN	No customer-initiated financial operation; only an explicit authorised unfreeze (trainer/admin script in this scope).
MATURED	Deposit accounts only; settlement/closure allowed.
CLOSED	Terminal; no new transactions. Closing requires zero balance (loan: fully repaid).

20. Session management

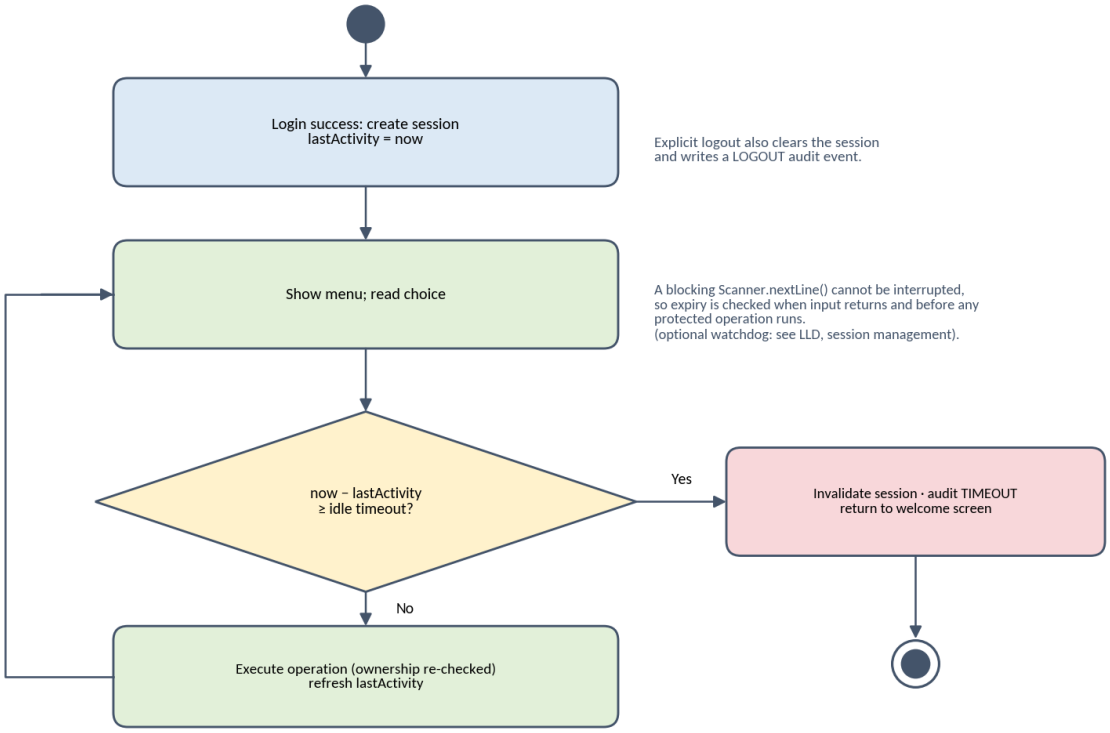


Figure 11. Session activity and timeout

- `SessionContext` holds `customerId`, `createdAt`, `lastActivity` (from an injectable `Clock`); default idle timeout 5 minutes (`session.idle.timeout.seconds`).
- Before every protected operation `SessionService.requireActive(session)` either refreshes `lastActivity` or throws `SessionExpiredException`, which returns the user to the welcome menu and audits TIMEOUT.
- Because `Scanner.nextLine()` blocks, expiry is detected when input returns. **Optional watchdog:** a daemon thread may poll the session and print a notice/exit the menu loop; if used, guard shared state with `volatile` or atomic fields and keep input reading non-blocking (`System.in.available()` polling).
- Logout clears the session reference, audits LOGOUT, and returns to the welcome screen. The password-change flow requires the current password again.

21. Validation matrix

Input / operation	Validation	Outcome
Registration	Required fields; email format; username pattern; password policy; unique username/email.	Reject invalid values; never persist a partial customer.
Login	Required fields; customer ACTIVE and not locked; hash verification.	Session only on success; generic failure message.
Money amount	Parses as BigDecimal; > 0; scale ≤ 2; ≤ configured limit.	Reject malformed, zero, negative, over-scale or over-limit values.
Account number	Resolves to an existing account; ownership checked server-side; status/type eligible.	Reject unknown, closed, frozen or unauthorized accounts.
Withdrawal	Balance sufficient after locking and re-read.	Reject with no change to balance or ledger.
Transfer	Source ≠ destination; both eligible; source owned.	No partial debit/credit; rollback on any failure.
Pagination	Page ≥ 0; page size 1–50; stable ordering.	Predictable pages; reject extremes.
CLI input	Empty strings; letters where numbers are required; EOF (Ctrl-D/Ctrl-Z); repeated invalid input (max 3 then return to menu).	Helpful message and a safe menu state; no stack trace.

22. Exception and error handling

```
BankingException (RuntimeException, base)
├── ValidationException            bad input, duplicates
├── AuthenticationException        bad credentials / locked  (generic message)
├── AuthorizationException         not the owner
├── SessionExpiredException        idle timeout
├── AccountNotEligibleException    closed / frozen / wrong type
├── InsufficientFundsException     balance too low
└── PersistenceException           SQL/connection failure (cause kept for logs only)
```

Exception	User-safe message	Logged (restricted)	Audit
ValidationException	Specific, field-level hint	WARN, no secrets	Optional
AuthenticationException	“Invalid username or password.”	INFO outcome only	login_audit FAILURE/LOCKED
AuthorizationException	“Account not found.” (no leakage)	WARN with customerId and accountId	transaction_audit REJECTED
InsufficientFundsException	“Insufficient funds.”	INFO	transaction_audit REJECTED
PersistenceException	“The operation could not be completed. Please try again.”	ERROR with stack trace	transaction_audit FAILED/ROLLED_BACK

23. Configuration

Key	Source	Default / note
DB_URL, DB_USERNAME, DB_PASSWORD	Environment variables	Required; never committed.
session.idle.timeout.seconds	app.properties	300
db.query.timeout.seconds	app.properties	10
tx.max.amount	app.properties	1000000.00
auth.max.failed.logins / auth.lock.minutes	app.properties	5 / 15
password.hash.algorithm / cost	app.properties	bcrypt / 12
history.max.page.size	app.properties	50

24. Logging and audit design

Event	Stored in	Fields
Login success / failure / locked / timeout / logout	login_audit	customer_id (if known), username_attempted, outcome, timestamp.
Deposit, withdrawal, transfer, repayment — success, rejection, failure, rollback	transaction_audit	customer_id, reference, event_type, outcome, failure_code, timestamp.
Technical errors	Application log (SLF4J/Logback)	Level, class, correlation reference, exception; no passwords, hashes, DB credentials, or full account numbers (mask to last 4).

25. Testing strategy

Level	Examples	Evidence
Unit	Validators; Money scale; eligibility rules; password policy; session expiry with fake Clock.	JUnit reports.
Service	Duplicate registration; ownership denial; insufficient funds; invalid transfer; loan repayment rules.	Fake DAOs / Mockito.
DAO integration	CRUD; FK and unique violations; ResultSet mapping; rollback.	Dedicated `banking_test` schema; repeatable setup/teardown (Testcontainers optional).
Transaction	Success; failure after debit; credit failure; ledger-insert failure; concurrent withdrawals/transfers.	Assert balances *and* ledger after commit/rollback.
Security	No plaintext in DB; invalid login; cross-customer access; SQL-injection strings; timeout and logout.	Automated tests plus live demo.
Acceptance	Fresh schema → register → login → create accounts → deposit → withdraw → transfer → history → timeout → logout.	README and demo checklist.

Essential test cases

ID	Scenario	Expected result
T01	Register valid customer	Customer created; encoded hash stored; no plaintext.
T02	Register duplicate username/email	Rejected; existing data unchanged.
T03	Login with correct password	Session created; customer menu shown.
T04	Login with wrong password / unknown user	Denied with the same generic message; failure audited.
T05	Five consecutive failures	Account temporarily locked; LOCKED audited.
T06	Request another customer's account	Authorization denied; no data leaked.
T07	Deposit positive amount	Balance and ledger increase consistently.
T08	Withdraw more than available	Rejected; balance and ledger unchanged.
T09	Transfer between eligible accounts	Source decreases and destination increases by the same amount; one SUCCESS row.
T10	Force failure after source debit	Rollback restores both balances; no SUCCESS row; failure audited.
T11	Transfer to closed/frozen account or to self	Rejected before commit; no balance change.
T12	Session idle beyond timeout	Session invalidated; next protected action requires login.
T13	Malformed or negative amount; amount with 3 decimals	Handled safely; no DB mutation.
T14	Concurrent withdrawals / opposite-direction transfers (A→B and B→A)	No overspend; no deadlock failure beyond bounded retry.
T15	History pagination boundary	No duplicate or omitted rows; stable order.
T16	Deposit account early withdrawal / after maturity	Refused before maturity; settle succeeds after.
T17	Loan repayment: partial, exact, over-payment	Balance reduces; exact closes; over-payment refused.
T18	SQL-injection strings in all text inputs	Treated as data; no SQL error or change.

26. Build and run guidance

```
Prerequisites: JDK 17+ (document one version), Maven 3.9+, MySQL 8+ or PostgreSQL 14+.
Dependencies (pom.xml): JDBC driver (mysql-connector-j or postgresql), bcrypt library (e.g. at.favre.lib:bcrypt)
                        or argon2-jvm, slf4j-api + logback-classic, junit-jupiter, mockito-core (test).

1. Create database/schema 'banking' (and 'banking_test') and a least-privilege user.
2. Run sql/<dialect>/schema.sql.
3. export DB_URL=... DB_USERNAME=... DB_PASSWORD=...
4. mvn test            # unit + integration tests against banking_test
5. mvn clean package   # builds the runnable jar
6. java -jar target/banking-cli.jar
Never commit credentials. Sample data must not contain real customers or reusable passwords.
```

27. Implementation plan

Milestone	Weeks 2–3 focus	Exit criteria / deliverable
P1-M1	Requirements, use cases, class and ER diagrams, repo, schema skeleton	Design document and Git repository; DB connects.
P1-M2	Registration, hashing, login, connection handling	Positive and negative login tests pass (Week 2 review).
P1-M3	Customer profile, account creation and listing (all four types represented)	Ownership and status/type validation work.
P1-M4	Deposit, withdrawal, balance enquiry	BigDecimal everywhere; invalid and insufficient-funds cases rejected (Week 3 interim).
P1-M5	Atomic transfer, rollback, history	Rollback and concurrency tests pass.
P1-M6	Session timeout, validation, exceptions, audit; Deposit and Loan workflows	No secrets logged; logout/timeout tested.
P1-M7	JUnit completion, README, demo preparation	Clean-setup reproducible; Week 3 final evaluation.

PART C

Domain-Driven Design (DDD)

Ubiquitous language, bounded contexts, aggregates, invariants and events

28. Approach and scope

P1 is small enough that full DDD ceremony would be over-engineering. This part therefore applies **strategic DDD** (shared language, subdomains, bounded contexts) and a **light tactical subset** (entities, value objects, aggregates, domain services, repositories, domain events) so that business rules live in the model and the code reads like the business. Contexts are *logical modules inside one process*, not separate services; this prepares the code for the later service/API project without committing to it.

Subdomain classification

Subdomain	Type	Why	Investment
Account rules and money movement (Ledger)	Core	Where correctness matters most: eligibility, balances, atomic transfers.	Rich domain model, thorough tests.
Identity and access	Supporting	Required, specific to the app, but not differentiating.	Clean service + tests; use a library for hashing.
Audit and logging	Generic	Standard concern.	Simple append-only tables and SLF4J.

29. Ubiquitous language

Use these terms identically in conversation, documentation, class names, method names, tables and tests.

Term	Meaning	Code / data
Customer	A registered person who can log in and own accounts.	Customer, customers
Credentials	Username plus password; only the hash is stored.	username, PasswordHash
Session	A period of authenticated activity that ends by logout or idle timeout.	SessionContext
Account	A customer-owned record of a particular type with a balance and status.	Account, accounts
Account type	Savings, Current, Deposit or Loan; decides allowed operations.	AccountType
Balance	Cash held (Savings/Current/Deposit) or amount owed (Loan).	Money balance
Money	An amount with exactly two decimals; always positive when used as an operation amount.	Money
Deposit (operation)	Adding money to an account. Not to be confused with a Deposit *account*.	TransactionType.DEPOSIT
Withdrawal	Removing money from an account with sufficient funds.	WITHDRAWAL
Transfer	Moving money between two eligible accounts in one atomic step.	TRANSFER
Repayment	Paying down a loan from an owned Savings/Current account.	LOAN_REPAYMENT
Ledger entry	The permanent record of a completed money movement.	BankTransaction
Reference number	Unique, system-generated identifier of a ledger entry.	TransactionReference
Frozen / Matured / Closed	Account lifecycle states with fixed rules.	AccountStatus
Audit record	Append-only record of an attempt or outcome; never edited.	LoginAudit, TransactionAudit

30. Bounded contexts and context map

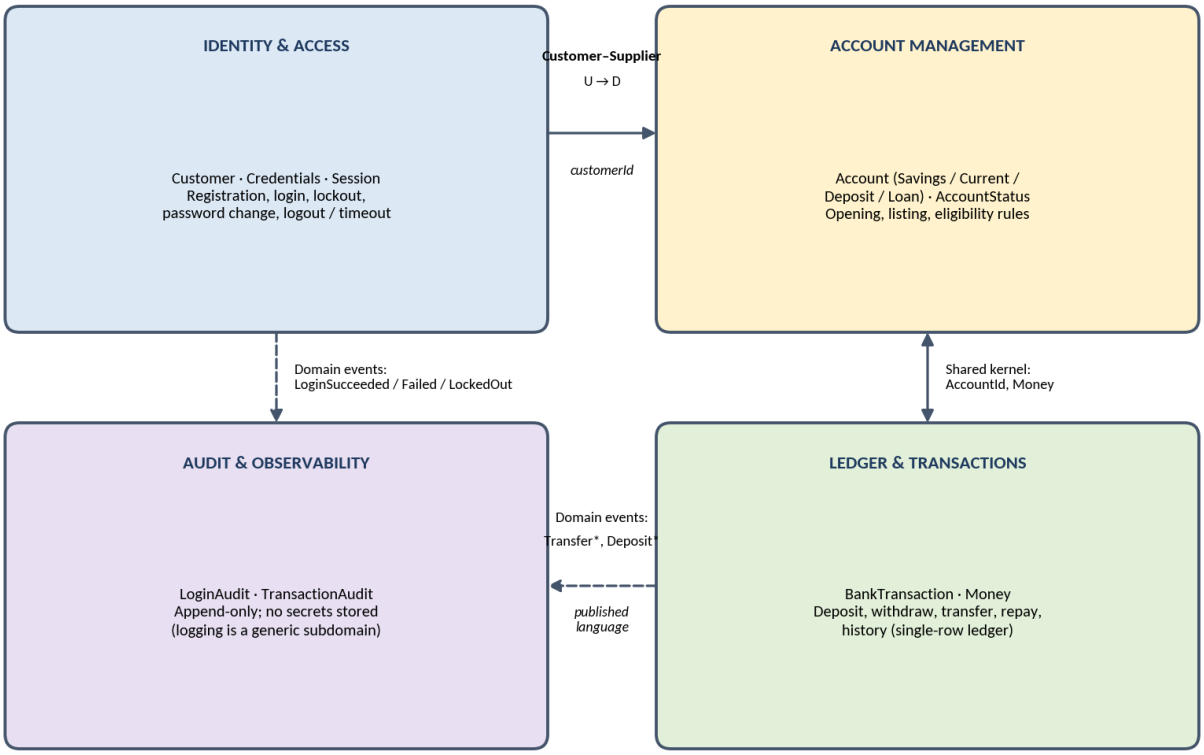


Figure 12. Bounded contexts and relationships

Context	Owns	Exposes	Relationship / integration
Identity & Access	Customer, credentials, session, lockout	customerId; LoginSucceeded/Failed/LockedOut events	Upstream of Account Management (Customer–Supplier): others consume only `customerId`.
Account Management	Account aggregate, eligibility and lifecycle rules	AccountId, AccountNumber, eligibility queries	Shares a small kernel (AccountId, Money) with Ledger; they evolve together.
Ledger & Transactions	BankTransaction, transfer coordination, history	Ledger entries; Deposit/Withdrawal/Transfer events	Calls Account Management to lock/mutate balances inside one DB transaction.
Audit & Observability	Login and transaction audit records	Append-only writers	Conformist subscriber to domain events; no business logic.

Translation to code. In P1 the contexts map to sub-areas of existing packages (for example `service.AuthService` for Identity, `service.AccountService` for Account Management, `service.TransactionService` for Ledger). Cross-context references use IDs only; a context never reaches into another's tables directly except through the service API.

31. Aggregates, entities and value objects

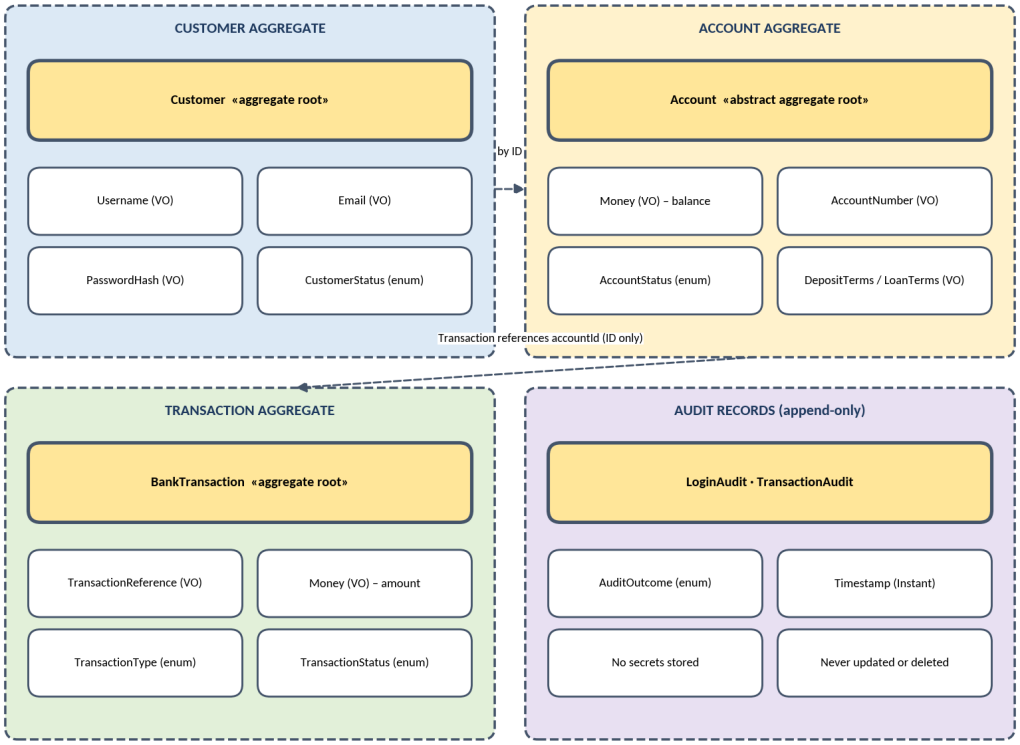


Figure 13. Aggregates (consistency boundaries)

31.1 Tactical building blocks

Element	Kind	Identity / equality	Invariants it guarantees
Customer	Aggregate root (entity)	customerId	Unique username/email; status transitions ACTIVE ↔ LOCKED → CLOSED; lockout rule.
Account (Savings, Current, Deposit, Loan)	Aggregate root (entity)	accountId / AccountNumber	balance ≥ 0; only ACTIVE accounts transact; type-specific `canDebit/canCredit`; one owner.
BankTransaction	Aggregate root (immutable entity)	transactionId / reference	Amount > 0; shape per type (source/destination presence); reference unique; never edited.
LoginAudit / TransactionAudit	Append-only entities	auditId	Never updated or deleted; no secrets.
Money	Value object	amount (scale 2)	Immutable; scale ≤ 2; arithmetic returns new instances; never negative for operation amounts.
AccountNumber	Value object	string	Format and checksum rule; generated, not typed.
TransactionReference	Value object	string	Unique, non-guessable (time + random).
Username, Email, PasswordHash	Value objects	value	Format validation at construction; hash never exposed via toString/logs.
DepositTerms, LoanTerms	Value objects	values	Term > 0; maturity date derived; principal > 0.
AccountType, AccountStatus, TransactionType, TransactionStatus, AuditOutcome	Enums	name	Closed sets mirrored by DB CHECK constraints.

31.2 Aggregate design rules applied

- Aggregates reference each other **by ID only** (Account holds `customerId`; BankTransaction holds account IDs).
- A transfer touches **two Account aggregates**. Strict DDD says one transaction per aggregate; here the application service deliberately coordinates both inside **one database transaction** with pessimistic locks, because the brief

requires atomicity. This is a documented, intentional exception (no eventual consistency or sagas in P1).

- Behaviour lives on the model: `account.debit(Money)` / `credit(Money)` enforce status, type and funds rules and return the new state; the DAO only persists it.
- The ledger row is created by a factory (`BankTransaction.transfer(...)`) so that invalid shapes cannot be constructed.

32. Domain services and policies

Domain service / policy	Responsibility
TransferPolicy	Decides whether (source, destination, amount) is allowed: owner, status, types, distinct accounts, limits.
AccountEligibilityPolicy	Per-type operation matrix (Section 5.1); maturity and repayment rules.
PasswordPolicy	Minimum length and character classes; rejects username-equal passwords.
LockoutPolicy	Failure threshold and lock duration.
ReferenceGenerator / AccountNumberGenerator	Produce unique references and account numbers (injected for deterministic tests).

Application services (`AuthService, AccountService, TransactionService`) are thin orchestrators: authorize, open the transaction, load aggregates through repositories, call domain behaviour, persist, publish events. They contain no rule logic that belongs to the model.

33. Domain events

Events are small immutable objects (past-tense names) published synchronously in-process after the primary unit of work; the audit context subscribes. They are an exercise in decoupling, not a messaging system.

Event	Raised when	Consumers
CustomerRegistered	Registration committed	Audit (optional)
LoginSucceeded / LoginFailed / AccountLockedOut	Authentication outcome	LoginAuditDAO
SessionExpired / LoggedOut	Timeout or explicit logout	LoginAuditDAO
AccountOpened	Account persisted	Audit (optional)
MoneyDeposited / MoneyWithdrawn	Commit succeeded	TransactionAuditDAO
TransferCompleted	Commit succeeded	TransactionAuditDAO
TransferRejected / TransferFailed	Validation failed / rollback	TransactionAuditDAO (written in a separate transaction)
LoanRepaid / LoanClosed	Repayment committed / balance reaches zero	TransactionAuditDAO

Important: failure events must be recorded *after* the money transaction has rolled back and in their own transaction; otherwise the audit row would be rolled back with the failed work.

34. Repositories and layering

DDD concept	P1 implementation	Package
Repository	DAO interfaces returning aggregates (`AccountDAO.lockForUpdate`); JDBC implementations map rows to domain objects.	dao, dao.jdbc
Aggregate / entity / VO	Plain Java classes with validation in constructors and factories.	model
Domain service / policy	Stateless classes with pure logic, easy to unit-test.	service (or model.policy)
Application service	Use-case orchestration and transaction boundary.	service
Anti-corruption	DTOs/requests at the menu boundary; ResultSet mapping isolated in DAOs.	app, dao
Domain events	Simple `EventPublisher` interface with an in-process implementation.	service / util

35. Business invariants catalogue

ID	Invariant	Enforced in	Test
INV-01	No plaintext or reversible password is ever stored.	PasswordHasher, schema column name/length	T01
INV-02	A customer can never read or change another customer's account.	Services + owner-scoped DAOs	T06
INV-03	Operation amounts are > 0 with scale ≤ 2.	Money, TransactionValidator, CHECK	T13
INV-04	A withdrawal or transfer never makes a balance negative.	Account.debit, locked re-read, CHECK	T08, T14
INV-05	A transfer is all-or-nothing.	Single DB transaction	T09, T10
INV-06	A SUCCESS ledger row exists only for committed money movement.	Insert inside the same transaction; failures only in audit	T10
INV-07	Loan balance is never spendable and never exceeds principal after repayment.	LoanAccount.canDebit=false; repayment rule	T17
INV-08	Deposit accounts release funds only after maturity.	DepositAccount rules	T16
INV-09	Frozen, matured (except settle) and closed accounts do not transact.	AccountEligibilityPolicy	T11
INV-10	An expired or logged-out session cannot perform protected operations.	SessionService.requireActive	T12
INV-11	Audit records are append-only and contain no secrets.	DAO (no update/delete); log review	Security tests
INV-12	References and account numbers are unique.	UNIQUE constraints	DAO tests

PART D

Acceptance, Governance and Traceability

Definition of done, evaluation, AI-assisted development and requirement coverage

36. Requirement-to-design traceability

Requirement	Design elements	Verification
F01–F02 Registration, login (P1-01, P1-02)	AuthService, CustomerDAO, PasswordHasher, LoginAuditDAO, LockoutPolicy	T01–T05; inspect stored hash
F03 Profile (P1-03)	CustomerService, CustomerDAO	Unit/service tests
F04–F05 Accounts (P1-04, P1-05)	Account hierarchy, deposit_details, loan_details, AccountService	T06, T16, T17; create/list multiple
F06–F07 Deposit/withdraw (P1-06, P1-07)	TransactionService, AccountDAO, TransactionDAO	T07, T08
F08 Transfer (P1-08)	Transaction boundary + row locking + retry	T09–T11, T14
F09–F10 Balance, history (P1-09, P1-10)	AccountService, TransactionDAO pagination	T15
F11 Session (P1-11)	SessionService, SessionManager, Clock	T12
F12 Integrity (P1-12)	Constraints, PreparedStatement, JdbcTransaction	T13, T18, DAO tests
F13 Audit (P1-13)	LoginAuditDAO, TransactionAuditDAO, domain events	Audit-table inspection tests
F14 Testing (P1-14)	Test strategy §25	JUnit report; coverage review

37. Definition of done and acceptance gates

P1 is ready for final evaluation when it builds from a clean checkout, schema setup is documented and reproducible, all four account categories operate under explicit rules, hashes (not plaintext) are stored, every protected operation enforces ownership, transfers are atomic under failure and concurrency, history is consistent, timeout/logout work, and automated tests cover success and failure paths.

Minimum gates (independent of total marks)

- No plaintext password storage.
- No cross-customer account access.
- No successful withdrawal exceeding available funds.
- No partial fund transfer when any step fails.
- No SQL built by concatenating untrusted input.
- Successful demonstration from a clean database using documented instructions.

38. Evaluation rubric (100 marks)

Area	Marks	What is assessed
Core Java, OOP and code quality	25	Class design, interfaces, collections, enums, maintainability.
JDBC, SQL and schema design	20	DAO separation, constraints, prepared statements, indexing.
Banking functionality and transaction integrity	25	Account operations, transfers, rollback, ledger consistency, account-type rules.
Exception handling, validation and security	15	Hashing, authorization, timeout, input validation, safe errors.
Unit testing, Git and documentation	15	Coverage, meaningful commits, README, reproducibility.
Total	**100**	

39. AI-assisted development and participant rotation

The cohort is split into two groups for P1 and P2. For an odd cohort the extra participant is allocated according to training policy.

Group	P1 — Secure Banking CLI	P2	Next assignment
A (≈50%)	AI-assisted development	AI-assisted development	Rotate to designated associates
B (≈50%)	Primarily independent development	Primarily independent development	Rotate to designated associates

- AI may generate boilerplate, explain JDBC errors, propose test cases and review code.
- Participants must verify AI-generated SQL, security logic and transaction behaviour.
- Each participant must explain password hashing, authorization and rollback during evaluation.
- AI-assisted contributions are recorded in the README or a short development log.
- Individual understanding is evaluated through live coding or a small change request (for example, block transfers to closed accounts, or add history filtering) during the final demo.
- Keep individual assessment records; individual competency must never depend on another associate's implementation.

40. Final submission checklist

Item	Item
[ ] Java source with layered packages	[ ] Transaction history and audit records
[ ] SQL schema (and optional fake sample data)	[ ] Session timeout and logout
[ ] Maven pom.xml and dependency notes	[ ] JUnit tests including transfer rollback and concurrency
[ ] Registration and secure login	[ ] README: setup, configuration, run, AI log
[ ] Savings, Current, Deposit and Loan support	[ ] Git repository with meaningful commit history
[ ] Deposit, withdrawal, transfer and repayment workflows	[ ] Demo of successful and failed scenarios

41. Limitations and future enhancements

Possible later work: configurable overdraft, deposit interest and early-closure penalties, loan schedules and disbursement, richer history search, statement export, optimistic versioning, audit retention, migration tooling (Flyway/Liquibase), MFA concepts, and a service/API layer in a later project (the bounded contexts in Part C are the natural seams). None of these should distract from the P1 objective: authentic Core Java, JDBC, SQL and sound transaction design.

— End of document —